



Department of Electrical & Electronic Engineering
Rajshahi University of Engineering & Technology

LAB Report

Course Code : **EEE 3100**

Course Title : **Electronics Shop Practice**

Experiment No. : 04

Experiment Name : Implementation of a Robotic System Using the L293D Motor Driver IC

Submitted By:

Name : Labib Marwan Hoque

Roll : 2101064

Section : B

Submitted To:

Asif Zaman Rizve

Lecturer

Department of Electrical & Electronic
Engineering

Rajshahi University of Engineering &
Technology

Date of Experiment : 28 April, 2025

Date of Submission : 12 May, 2025

Experiment No. 04

Experiment Title: Implementation of a Robotic System Using the L293D Motor Driver IC

Objectives:

- To study the principle and operation of a L293D Motor Driver IC.
- To design a a robotic system using the L293D Motor Driver IC with the help of Arduino microcontroller.

Theory:

The implementation of robotic systems often requires precise control of DC motors, which are the primary actuators used to produce movement. However, microcontrollers like the Arduino cannot directly drive these motors due to current limitations. The **L293D Motor Driver IC** serves as a crucial interface between the low-power control circuit and the high-power motors. It acts as a bridge that allows a microcontroller to control the direction and speed of motors safely and efficiently.



Figure 4.1: L293D Motor Driver IC

1. Overview of L293D Motor Driver IC:

The L293D is a dual H-Bridge motor driver IC, which means it can drive two DC motors independently in both forward and reverse directions. It operates on a 4.5V to 36V supply range for the motor and has separate logic and motor supply pins. The L293D can supply a maximum current of 600mA per channel, which is suitable for most small DC motors used in educational and hobby-grade robotic systems.

Internally, each H-Bridge is composed of transistor pairs arranged in such a way that changing the input logic levels can reverse the polarity of the motor voltage. This capability is essential for robotic systems that need bidirectional control.

2. Pin Configuration and Functionality

The IC consists of 16 pins with the following key connections:

- **Input Pins (IN1, IN2, IN3, IN4):** Receive logic signals from the microcontroller.
- **Output Pins (OUT1, OUT2, OUT3, OUT4):** Connected to the motor terminals.
- **Enable Pins (EN1, EN2):** Control whether each motor is enabled. These must be HIGH to allow motor operation.
- **VCC1 (Logic Voltage):** Typically 5V, powers the internal logic circuit.
- **VCC2 (Motor Voltage):** Powers the motors (up to 36V).
- **GND:** Common ground for both logic and motor power.

The basic truth table for motor control using one H-Bridge of the L293D is shown below:

IN1	IN2	Motor Direction
1	0	Forward
0	1	Reverse
1	1	Brake
0	0	Stop

3. Working Principle

When a HIGH signal is sent to an input pin, the corresponding output pin goes HIGH (assuming the enable pin is also HIGH). By adjusting the combination of HIGH and LOW signals at the input pins, one can control the rotation direction of the motors. This technique is particularly useful for differential drive robotic systems, where varying the direction and speed of two motors allows for forward movement, reverse movement, and turning.

4. Integration with Arduino for Robotic Control

In a robotic system, the L293D is connected between the Arduino and the DC motors. The Arduino sends digital HIGH/LOW signals to the input pins of the L293D. The enable pins can be tied to HIGH for basic control or connected to PWM outputs of the Arduino to achieve speed control via pulse-width modulation (PWM).

For example, to rotate a robot forward:

- Left Motor: IN1 = HIGH, IN2 = LOW
- Right Motor: IN3 = HIGH, IN4 = LOW

To turn left:

- Left Motor: IN1 = LOW, IN2 = HIGH
- Right Motor: IN3 = HIGH, IN4 = LOW

The robotic platform can thus navigate through the environment by altering motor signals dynamically based on sensor input or programmed behavior.

The L293D motor driver IC is an essential component in implementing robotic systems that require directional and speed control of DC motors. Its robust functionality, ease of integration with microcontrollers, and capacity to handle typical motor voltage and current needs make it an ideal choice for educational and hobby robotic applications.

Required Apparatus:

Table 4.1: Table For Required Apparatus

SI No.	Component Name	Specification	Quantity
1	Motor Driver	L293D Motor Driver IC	1
2	DC Gear Motor	6V–12V DC gear motor suitable for robotics	2
3	Pushbutton	-	1
4	Micro-controller Board	Arduino Uno	1
5	Power Supply	Variable DC (15V max)	1
6	Jumper Wires	Male-to-male, male-to-female jumper wires	As per need
7	Breadboard	For prototyping the circuit connections	1

Circuit Diagram:

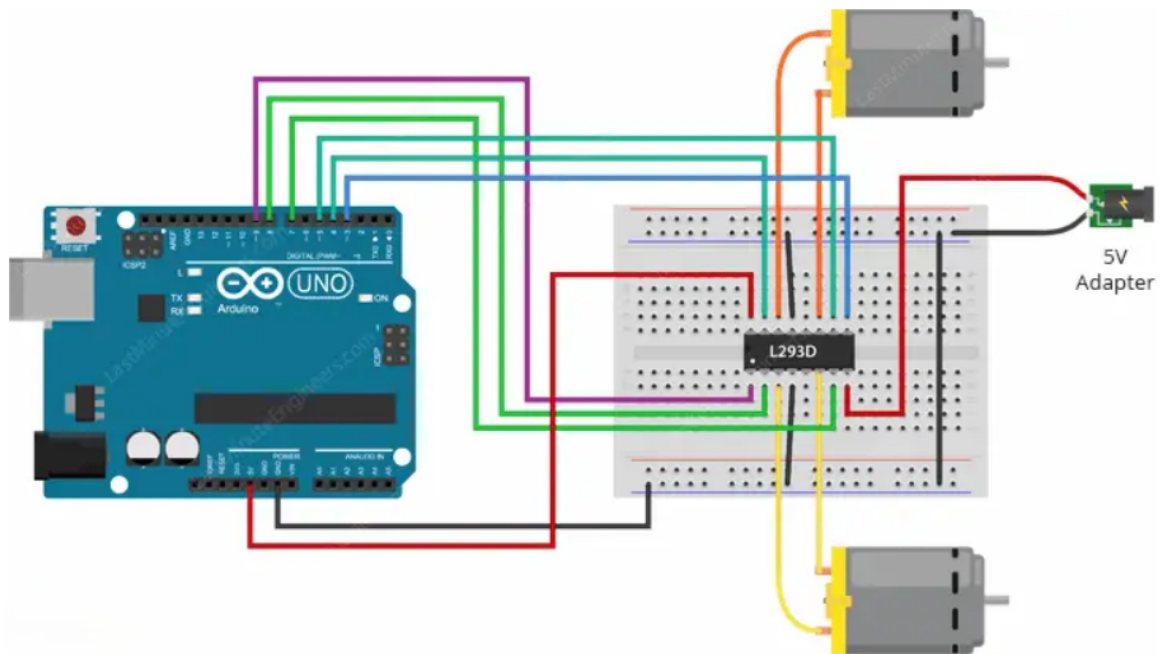


Figure 4.2: Schematic Diagram

Experimental Setup:

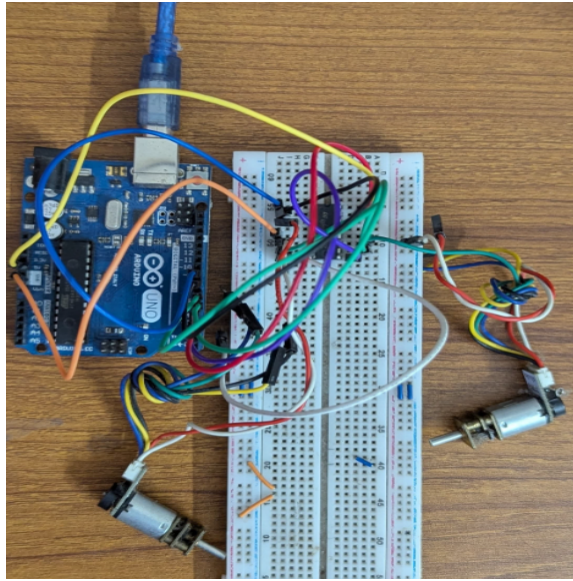


Figure 4.3: Experimental Setup

Code:

```
// Motor A connections
int enA = 9;
int in1 = 8;
int in2 = 7;

// Motor B connections
int enB = 3;
int in3 = 5;
int in4 = 4;

void setup() {
    // Set all the motor control pins to outputs
    pinMode(enA, OUTPUT);
    pinMode(enB, OUTPUT);
    pinMode(in1, OUTPUT);
    pinMode(in2, OUTPUT);
    pinMode(in3, OUTPUT);
    pinMode(in4, OUTPUT);

    // Turn off motors - Initial state
    digitalWrite(in1, LOW);
    digitalWrite(in2, LOW);
    digitalWrite(in3, LOW);
    digitalWrite(in4, LOW);
}

void loop() {
    directionControl();
    delay(1000);
    speedControl();
    delay(1000);
}
```

```

}

// This function lets you control spinning direction of motors
void directionControl() {
    analogWrite(enA, 255);
    analogWrite(enB, 255);

    digitalWrite(in1, HIGH);
    digitalWrite(in2, LOW);
    digitalWrite(in3, HIGH);
    digitalWrite(in4, LOW);
    delay(2000);

    digitalWrite(in1, LOW);
    digitalWrite(in2, HIGH);
    digitalWrite(in3, LOW);
    digitalWrite(in4, HIGH);
    delay(2000);

    digitalWrite(in1, LOW);
    digitalWrite(in2, LOW);
    digitalWrite(in3, LOW);
    digitalWrite(in4, LOW);
}

// This function lets you control speed of the motors
void speedControl() {
    digitalWrite(in1, LOW);
    digitalWrite(in2, HIGH);
    digitalWrite(in3, LOW);
    digitalWrite(in4, HIGH);

    for (int i = 0; i < 256; i++) {
        analogWrite(enA, i);
        analogWrite(enB, i);
        delay(20);
    }

    for (int i = 255; i >= 0; --i) {
        analogWrite(enA, i);
        analogWrite(enB, i);
        delay(20);
    }

    digitalWrite(in1, LOW);
    digitalWrite(in2, LOW);
    digitalWrite(in3, LOW);
    digitalWrite(in4, LOW);
}

```

Experimental Procedure:

For the basic (main) experiment, the following steps were executed:

1. Connected the L293D motor driver IC to the Arduino UNO as per the circuit diagram.

2. Connected Motor A to pins IN1 (8) and IN2 (7), and Motor B to pins IN3 (5) and IN4 (4) of the Arduino.
3. Connected the enable pins ENA (9) and ENB (3) to the PWM-capable pins of Arduino for speed control.
4. Uploaded the Arduino code to the board using the Arduino IDE.
5. In the `setup()` function, set all motor-related pins as OUTPUT and turned motors off initially.
6. In the `loop()` function, first executed the `directionControl()` function to rotate both motors in one direction and then reverse.
7. Used the `speedControl()` function to gradually increase and decrease motor speed using PWM signals.
8. Observed motor behavior during directional and speed control phases.

Hands-on Learning Projects for Mastery of DC Motor Control with Arduino and L293D

Task No.: 01

Task Name: Designing Timed Sequential Control of Dual Motors in a Repetitive Cycle

Objective:

- To implement a time-based motor control system where Motor 1 runs for a specified duration, followed by Motor 2, and then both motors run together, forming a repeatable operational cycle.

Code:

```
// Motor A connections
int enA = 9;
int in1 = 8;
int in2 = 7;

// Motor B connections
int enB = 3;
int in3 = 5;
int in4 = 4;

void setup() {
    // Set all motor control pins to outputs
    pinMode(enA, OUTPUT);
    pinMode(enB, OUTPUT);
    pinMode(in1, OUTPUT);
    pinMode(in2, OUTPUT);
    pinMode(in3, OUTPUT);
    pinMode(in4, OUTPUT);
}
```

```

    // Turn off motors initially
    digitalWrite(in1, LOW);
    digitalWrite(in2, LOW);
    digitalWrite(in3, LOW);
    digitalWrite(in4, LOW);
}

void loop() {
    // Run Motor A for 5 seconds
    analogWrite(enA, 255);
    digitalWrite(in1, HIGH);
    digitalWrite(in2, LOW);
    delay(5000);
    digitalWrite(in1, LOW);
    digitalWrite(in2, LOW);

    // Run Motor B for 5 seconds
    analogWrite(enB, 255);
    digitalWrite(in3, HIGH);
    digitalWrite(in4, LOW);
    delay(5000);
    digitalWrite(in3, LOW);
    digitalWrite(in4, LOW);
    delay(1000);

    // Run both motors for 10 seconds
    analogWrite(enA, 255);
    analogWrite(enB, 255);
    digitalWrite(in1, HIGH);
    digitalWrite(in2, LOW);
    digitalWrite(in3, HIGH);
    digitalWrite(in4, LOW);
    delay(10000);

    // Stop all motors
    digitalWrite(in1, LOW);
    digitalWrite(in2, LOW);
    digitalWrite(in3, LOW);
    digitalWrite(in4, LOW);
    delay(1000);
}

```

Task No.: 02

Task Name: Implementing Conditional Activation in Timed Dual Motor Control

Objective:

- To design a control system where Motor 2 activates only upon receiving a user input or signal after Motor 1 stops, followed by simultaneous motor operation for a fixed duration, in a continuous cycle.

Code:


```

// Motor A connections
int enA = 9;
int in1 = 8;
int in2 = 7;
// Motor B connections
int enB = 3;
int in3 = 5;
int in4 = 4;
bool a = true;

void setup() {
    // Set all the motor control pins to outputs
    pinMode(enA, OUTPUT);
    pinMode(enB, OUTPUT);
    pinMode(in1, OUTPUT);
    pinMode(in2, OUTPUT);
    pinMode(in3, OUTPUT);
    pinMode(in4, OUTPUT);
    pinMode(12, INPUT);
    // Turn off motors - Initial state
    digitalWrite(in1, LOW);
    digitalWrite(in2, LOW);
    digitalWrite(in3, LOW);
    digitalWrite(in4, LOW);

void loop() {
    if (a == true) {
        analogWrite(enA, 255);
        digitalWrite(in1, HIGH);
        digitalWrite(in2, LOW);
        delay(5000);
        a = false;
        digitalWrite(in1, LOW);
        digitalWrite(in2, LOW);
    }
    if (digitalRead(12) == HIGH) {
        analogWrite(enB, 255);
        // Motor 2 run 5 sec
        digitalWrite(in3, HIGH);
        digitalWrite(in4, LOW);
        delay(5000);
        digitalWrite(in3, LOW);
        digitalWrite(in4, LOW);
        delay(1000);

        // Now run both motors for 10 seconds
        analogWrite(enA, 255);
        analogWrite(enB, 255);
        digitalWrite(in1, HIGH);
        digitalWrite(in2, LOW);
        digitalWrite(in3, HIGH);
        digitalWrite(in4, LOW);
        delay(10000);

        digitalWrite(in1, LOW);
        digitalWrite(in2, LOW);
        digitalWrite(in3, LOW);
        digitalWrite(in4, LOW);
    }
}

```

```

        delay(1000);
        a = true;
    }
}

```

Task No.: 03

Task Name: Developing an Interlock-Based Safety Control in Dual Motor Sequencing

Objective:

- To integrate a safety mechanism into the motor control sequence where the entire process halts upon detecting a danger signal, while preserving conditional and timed motor operation.

Code:

```

// Motor A connections & Motor B connections
int enA = 9;
int in1 = 8;
int in2 = 7;
int enB = 3;
int in3 = 5;
int in4 = 4;
const int dangerPin = 12; // Danger button pin
bool stageA_done = false;
bool paused = false;
unsigned long lastTime = 0;
int state = 0;

void setup() {
    // Set motor pins as outputs
    pinMode(enA, OUTPUT);
    pinMode(enB, OUTPUT);
    pinMode(in1, OUTPUT);
    pinMode(in2, OUTPUT);
    pinMode(in3, OUTPUT);
    pinMode(in4, OUTPUT);
    pinMode(dangerPin, INPUT);
    stopAllMotors();
}

void loop() {
    // Check danger signal
    if (digitalRead(dangerPin) == HIGH) {
        stopAllMotors();
        paused = true;
        return; // Exit loop early if paused
    }
    if (paused) {
        // Wait until dangerPin is LOW to resume
        if (digitalRead(dangerPin) == LOW) {
            paused = false;
            delay(200); // Debounce
        } else {
            return;
        }
    }
}

```

```

    }
}
switch (state) {
    case 0: // Run Motor A for 5 sec (only once per full cycle)
        analogWrite(enA, 255);
        digitalWrite(in1, HIGH);
        digitalWrite(in2, LOW);
        lastTime = millis();
        state = 1;
        break;

    case 1: // Wait while Motor A runs
        if (millis() - lastTime >= 5000) {
            digitalWrite(in1, LOW);
            digitalWrite(in2, LOW);
            state = 2;
        }
        break;

    case 2: // Wait for danger button press to begin next phase
        if (digitalRead(dangerPin) == HIGH) {
            paused = true;
            return;
        }

        // Now run Motor B for 5 sec
        analogWrite(enB, 255);
        digitalWrite(in3, HIGH);
        digitalWrite(in4, LOW);
        lastTime = millis();
        state = 3;
        break;

    case 3: // Wait while Motor B runs
        if (millis() - lastTime >= 5000) {
            digitalWrite(in3, LOW);
            digitalWrite(in4, LOW);
            delay(1000);
            state = 4;
        }
        break;

    case 4: // Run both motors for 10 sec
        analogWrite(enA, 255);
        analogWrite(enB, 255);
        digitalWrite(in1, HIGH);
        digitalWrite(in2, LOW);
        digitalWrite(in3, HIGH);
        digitalWrite(in4, LOW);
        lastTime = millis();
        state = 5;
        break;

    case 5: // Wait while both motors run
        if (millis() - lastTime >= 10000) {
            stopAllMotors();
            delay(1000);
            state = 0; // Reset for next cycle
        }
}

```

```
        break;
    }
}

void stopAllMotors() {
    digitalWrite(in1, LOW);
    digitalWrite(in2, LOW);
    digitalWrite(in3, LOW);
    digitalWrite(in4, LOW);
}
```

Discussion:

In this lab, creating a basic robotic system using the L293D motor driver IC and Arduino microcontroller was the main objective. It was successfully done. After that, another three robotic system were built according to the task objectives. Task no. 2 was executed in the lab where only one motor will rotate for a moment, then another motor will start to rotate for a moment until a push button is pressed, at last, both will rotate for a moment. It was also done successfully. Remaining two tasks were performed personally in household successfully.

Justification of CO2 and PO(j):

The first code demonstrates a simple timed operation for two DC motors, where Motor A is run for 5 seconds, followed by Motor B for another 5 seconds, and finally both motors are operated together for 10 seconds. The design is straightforward, using delay functions to control the duration of each motor's operation in a fixed sequence.

The second code introduces conditional logic by allowing user input to influence motor behavior. Here, Motor A runs automatically once, while the operation of Motor B and the running of both motors depend on a button press (connected to digital pin 12). This setup is done to influence the motor sequence externally, making the system more interactive.

The third code represents a more safety-aware design. A “danger” button is constantly monitored, and if pressed, it immediately stops the motor and prevents further operation until it is released. Instead of using blocking delays, the millis() is used to function always without stopping the micro-controller. It enables continuous input monitoring.

Conclusion:

In this experiment, we successfully implemented a robotic system using the L293D motor driver IC and Arduino UNO. The motors were controlled for both direction and speed using PWM signals. The system responded accurately to the programmed instructions, demonstrating the effective use of the L293D for DC motor control. This setup forms an important step in understanding motor driver applications in robotics.

References

- [1] <https://lastminuteengineers.com/l293d-dc-motor-arduino-tutorial/>